

TALLINN UNIVERSITY OF TECHNOLOGY
IT College

Marko Linde 176292IDDR

PRODUCTION MANAGER APP

Distributed systems (ICD0009)

Instructor: Andres Käver

Tallinn 2020

Author declaration

The purpose of the project is to make a simple and user friendly, easily understandable and partly automated production planning tool. It is meant for small production companies (some of my own friends) and gives an opportunity to offer a cheaper alternative for MS AX.

Abbreviations and definitions

DTO	<i>Data transfer object</i> , object, that is meant for keeping data without functionality
C#	<i>C sharp</i> , third generation programming language
REST	<i>Representational state transfer</i> , application communication standard which separates client from the server.
MVC	<i>Model-View-Controller</i> , architectural pattern, that separates API to structural logical components. <i>Model</i> represents program logical component, <i>View</i> represents user component and controller the interface between user and server
API	<i>Application programming interface</i> , interface which helps the program to communicate to outside world
DAL	<i>Data Access Layer</i>
BLL	<i>Business Logic Layer</i>
JSON	<i>JavaScript Object Notation</i> , structure for data transfer
JWT	<i>JSON web token</i> ,
UI	<i>User interface</i> ,
DOM	<i>Document Object Model</i> , Html API
UX	<i>User experience</i> ,
CRUD	<i>Create-Read-Update-Delete</i> ,
CI	<i>Continuous Integration</i> ,
E2E	<i>End to End</i> , testing structure
POP	<i>Page Object Pattern</i> , testing pattern, where the html components or pages are kept inside objects.
SOLID	
UOW	
FP	
GIT	
AOP	

Table of Contents

1 Assignment	5
2 Overview	6
2.1 Client side business requirements.....	6
2.2 User interface.....	7
2.3 Main user functionality preliminary	9
2.4 User functionality	9
3 Soft delete and update.....	9
3.1 Soft delete with in between tables	10
3.2 Creating surrogate keys with multiple values.....	12
3.3 Creating soft deletion with self reference.....	13
3.4 Historic tables	14
3.5 Conclusion	14
4 DAO versus Repository pattern.....	14
Annex 1 – DB Model.....	16
Annex 2.....	17

1 Assignment

Web service, where the program server component is written on C#. The application should work as a calendar, where the user can plan universal production cycle from smaller components production to component compilation and packaging.

Production Manager APP MVC has to be able to notify the user of problems via by itself and by other sub-users. Preferably be able to communicate with warehouse. Retrieve and send data to users. Give current information on production progress and give solution to user or automatically solve a problem. Different APP components provide accurate analysis about past production cycles and compare previous similar cycle to current solution.

Production Manager APP v0.1 and this project scope includes planning and communication between different production parties.

Future aspect includes mobile API for easier component utilization and notification management and reaction. API must have the possibility to synchronize data between several different warehouses. Possible future solution should include export and import of warehouse component list.

2 Overview

Production Manager APP v0.1 gives the production workflow planning and communication between designer and different production teams.

Server side DTO-s will be constructed from visual ERD-CASE diagram and database update and initialization will come through migration. Code first script files for database generation will be included at the end of this document.

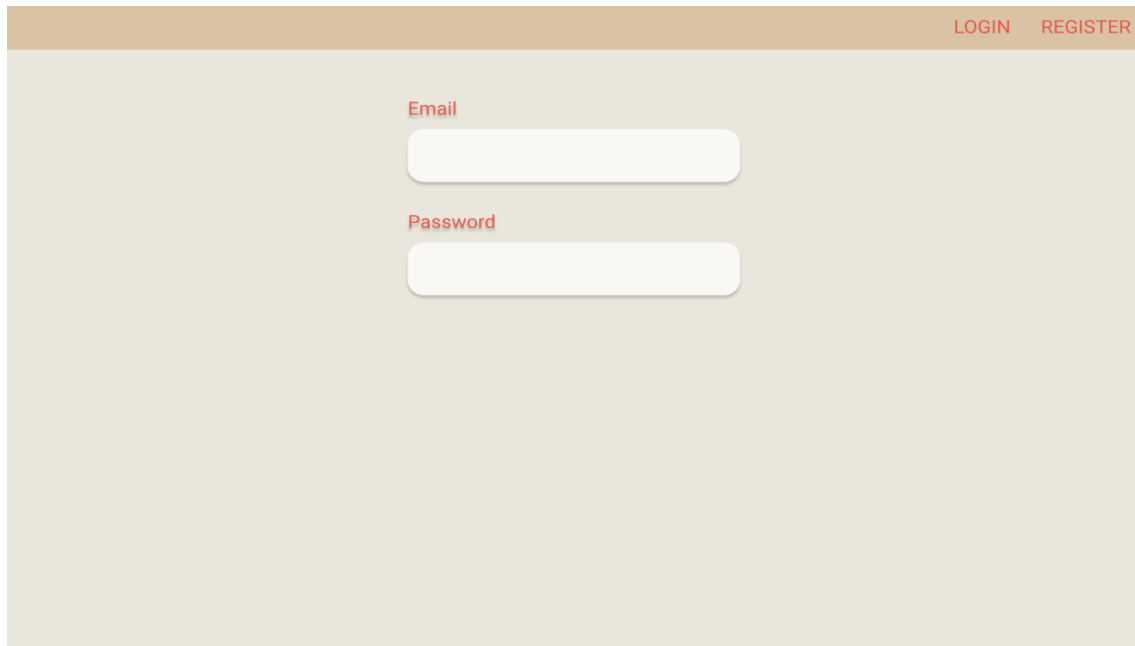
2.1 Client side business requirements

- program has to be written in C# language using Net Core 5.0. Class libraries have to use 5.0 aswell.
- program has to follow clean SOLID architecture and MVC pattern
- program features have to be layered for easy change.
- Data will be stored using MSServer and using taltech provided schema.
- Entity Framework Core will be used as wrapper over database and takes care of information mapping and virtual structure generation of DAL DTO-s
- DAL layer
- BLL layer
- API layer
- user creation and user activities basics are left for .Net individual authentication and identitycontext.
- User roles are segregated between Super, Main and Regular
- Role groups are segregated between Super user created Teams
- user client authorization will be with stateless JWT. JWT will get a secret which will expire with new key. JWT will also be added to every user request.

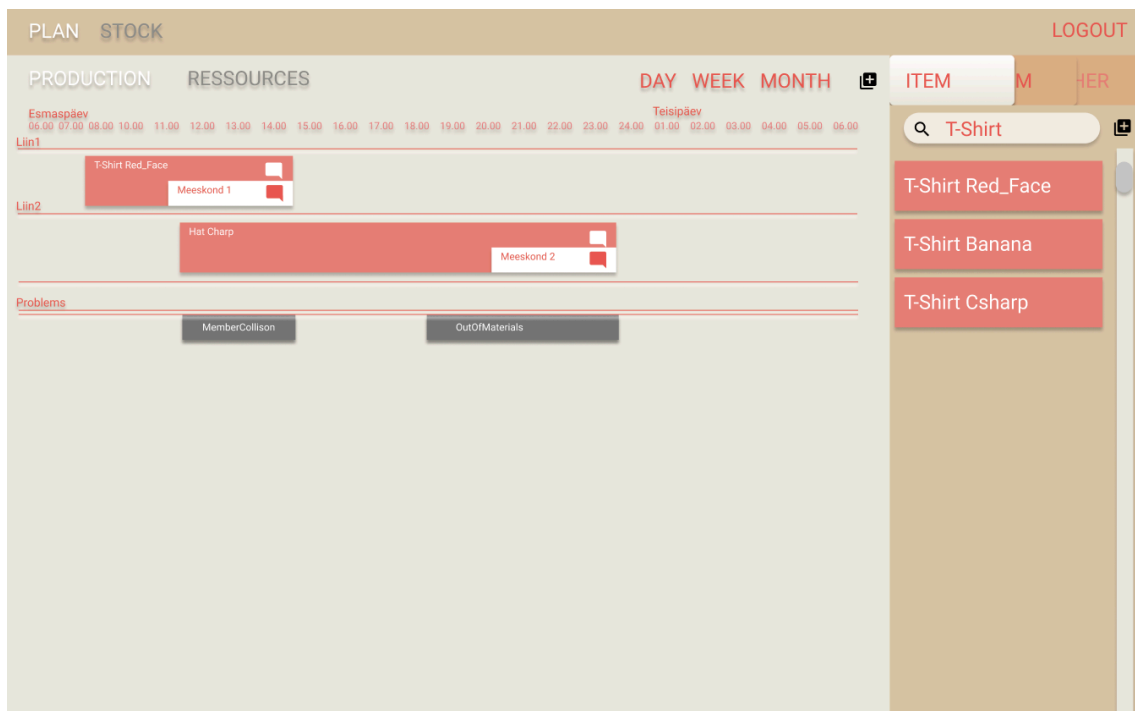
2.2 User interface

Main views made in Figma and interactive link:

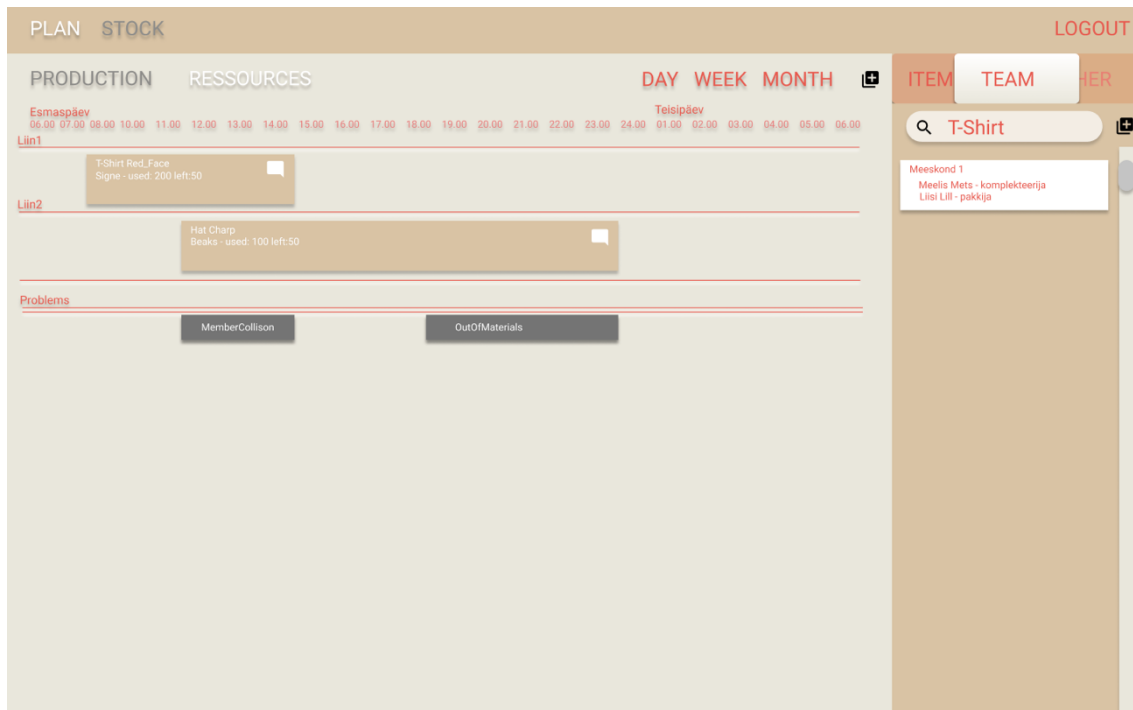
<https://www.figma.com/proto/B3btV75evn32c4pV20fqPO/ProductionAPP?node-id=31%3A26&scaling=min-zoom>



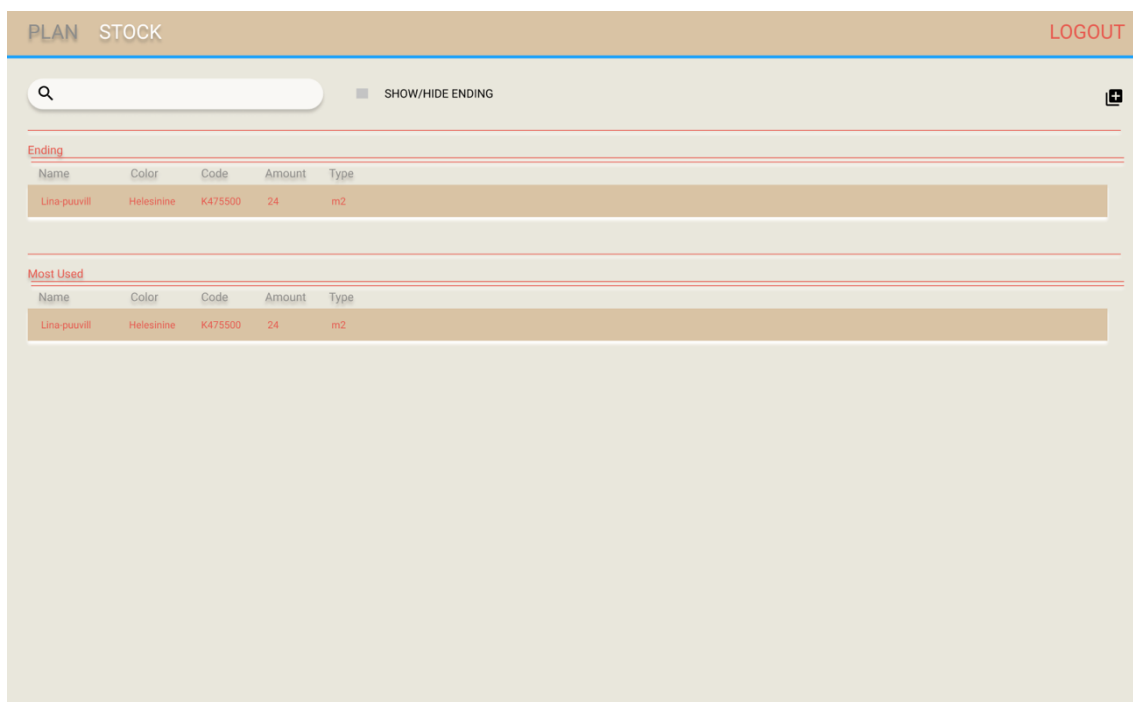
Drawing 1. Login screen



Drawing 2. Production plan screen



Drawing 3. Resource plan screen



Drawing 4. Resource overview

- Server-client communication datatype will be JSON
- UI development for lesser mistakes will be held in typescript

- UI and some UX will be configured by Materialize library.

2.3 Main user functionality **preliminary**

- ROLE based authentication

2.4 User functionality

- notifications CRUD
- fulfilling orders, marking extensions and problems

3 Soft delete and update

Managing data integrity and historical reference can be quite difficult to achieve. Considering that every business has their own features and work cycles, there cannot be a universal way of doing things. There can be mostly accepted storage patterns, but even then, it requires some thought if it is necessary to use soft delete.

Let's consider financial transactions and accounting, where nothing can be deleted and updated. Updating means that counter data is inserted and new data is created. Old data will still exist as valid data and there is no delete at all. So, if this is achieved by just new insertions to database, why would we need soft delete? A simple example: if a person changes their name, should we have a need for past record of its credentials? We can answer yes, if this person was a criminal, but this shows that the architect and customer must really consider if it is necessary and needed.

The first things with soft delete that usually come to mind are database integrity and information analytics. Even database integrity is somewhat arguable, because every data sensible company uses backups. Some of the companies doing soft deletion are asking themselves – we have all this data and what can we do with it and some ask if we can sell it in some formatted way. But still it is a good and faster way to revert to older data if something goes wrong.

Let's look at a single table (Drawing 1), where data can be inserted with primary key, created_at and modified_at (representing datetime property). The modified_at means both deletion and update. When it is filled, it represents old data set.

FOO

ID	MODIFIED_AT	CREATED_AT
1	NULL	2000

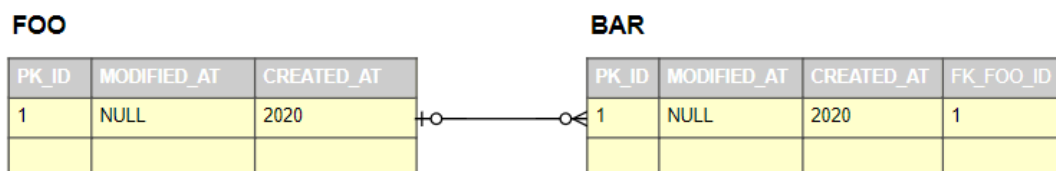
Drawing 5. Single database table.

With one table it is relatively understandable, where deleted row modified_at will point to new created_at row field. There are two ways of achieving this. Either we first modify the new row and then create the new row or copy the old row as deleted row and change the current row. The positive side of the last idea is that active data id does not need to be changed and if our purpose was to change the data of that single table and not the data of children the integrity of database would not change.

The difficult part is keeping the full integrity of master-child data relationship.

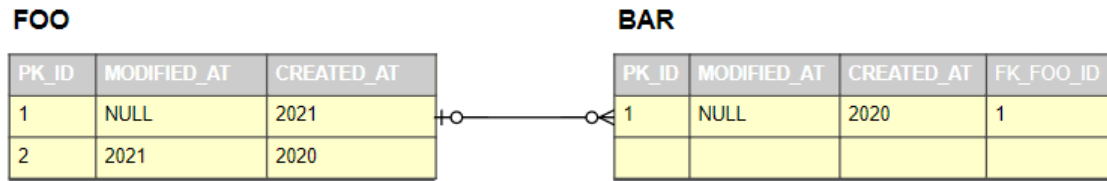
3.1 Soft delete with in between tables

One of the most common relationship between two tables is one to many (Drawing 2). Table FOO has zero or many items in table BAR. Table BAR item has one master item or nothing in table FOO.



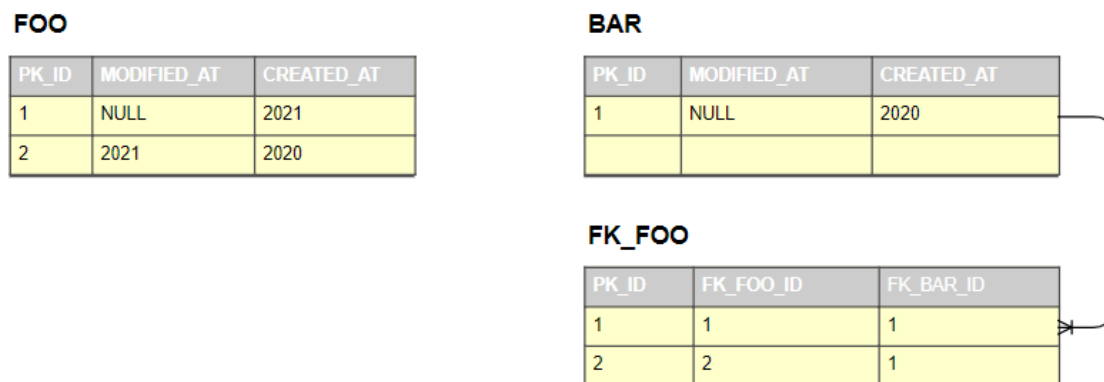
Drawing 6. One to many relationship

Updating data in FOO only, results in full integrity current state, but past data loses all its relationships to its children (Drawing 3).



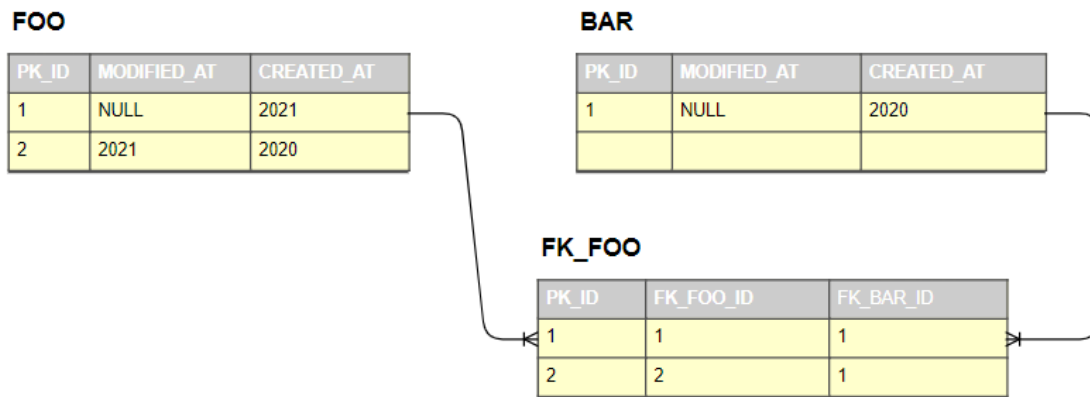
Drawing 7. PK_ID 2 broken past relationship

In this situation is the main problem how to preserve modified FOO PK_ID 2 relationship to table BAR. One way to achieve this is to make both FOO rows point to the same BAR row which will result with additional fields in not desirable many to many relationships. Leaving out the last idea would mean that additional key needs to be added to table BAR, keeping separate table for FOO keys, leaving us with a broken constraint (Drawing 4). Relationship between FOO and BAR will be broken, because we can't add multiple foreign keys to a table. We could add them to nullable separate foreign key fields, but this would mean, that every update and delete would need a separate field.



Drawing 8. Broken relationship between FOO and BAR

On closer observation we would see, that the table of FK_FOO is nothing more than in-between table to FOO and BAR (Drawing 5).

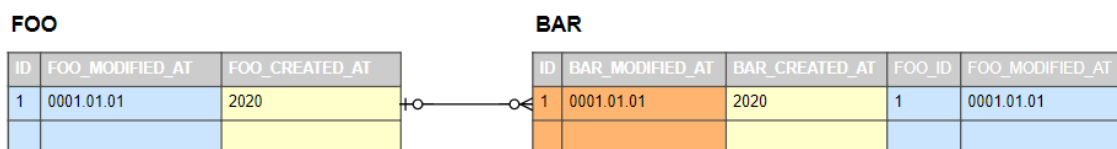


Drawing 9. In-between table

The relationships are fixed and BAR foreign key constraint would be replaced by a table. This is needed to be done with all the table relationships ending with 50% more tables than intended. Although it would not be easy to implement, it will have relatively clean table information, but historical data would still affect performance, because database table is still indexing and not ignoring updated and deleted rows. At first sight it would seem that the soft delete problem is solved but creating a one to one relationship in similar matter is impossible. Another negative side is that the models would be not directly relating to database structure.

3.2 Creating surrogate keys with multiple values

Another way of solving soft delete question is with combining multiple table values as surrogate keys. It is a working way of keeping historical changes and database integrity intact. In example primary keys have been constructed with id and <table>_modified_at values. Since primary keys can never be null or undefined, the modified_at values in example must be set in data entry (Drawing 6).



Drawing 10. Surrogate key values from id and <table>_modified_at

Now updating FOO table data would mean that children data must be renewed as well.

If only FOO is copied and inserted, the new entry will point to older self, but it will still be broken relative to children. So, all the data must be copied and entered again with the `<table>_modified_at` values closed and the old `created_at` values refreshed to the previously `modified_at` values. This will ensure us the full integrity of database. Updating BAR is simpler, because the only thing that has to be done is to copy the child table value and update the old value. Dates would point to timely modification and integrity is correct. Data deletion will be even simpler with update cascade, where all the children `modified_at` values will be automatically changed.

Problems will arise with one to one relationship. Deleting data in either parent or child table would mean the update of all database values, because one to one foreign key has to be unique and this can not be changed. One way would be to ignore the soft delete and really delete some data or use some other methods described in next chapters.

3.3 Creating soft deletion with self reference

This solution will add more problems to one to many relationship, because as in previous solution, updating parent entry requires coping of all data from all the tables, inserting it under the new entry and closing the old entry. So, it will introduce a new semantic child foreign key field to entity that references to its parent. And the query would require an in-between table to collect all the parent children data, for the correct information. The only thing it does, is add complexity to queries and data collection.

In previous chapter we talked about one to one relationship, where all the data would have to be copied and inserted again. This solution would be one way to solve one to one relationship. The logic behind it is simple, because it will always be the last entry that represents the newest data. The idea of one to one relationship needs the foreign key of child to be unique. Meaning that already entered parent surrogate key can not be entered again. Updating and deleting parent entity represents no problems, because updating means new entry of old copy and this will have new foreign key and the deleting means just closing the `modified_at` table value. But updating one to one child is very problematic, because we can not enter the same parent foreign key again. Although foreign key can not be entered again, it is possible to add additional child reference to itself. Here updating a child means copy of old self and entry of the data under itself with new updated value. The parent foreign key is not needed to be entered again, because it

exists at first child that points to updated second child. Another problem arises after the first child update, because the new entry will have null values as foreign keys, thus it will not be possible to update again. These issues can be solved with changing the default modified_at entry. As described in previous chapters, where modified_at entry has entered dummy data as 0001.01.01 00:00:00.000, it is possible to do the same thing here by creating default value in table creation where the very big or small date is incremented by millisecond to 0001.01.01 00:00:00.001. Thus, fooling the unique foreign key requirement. Now deleting the one to one child is doable as well. The tricky part is not deleting but creating new child element for the parent. It is doable by adding new element to the already entered parent foreign key tree. It is not the most elegant and simplest solution, but it will work.

3.4 Historic tables

This is the best solution for soft deletion and that is why most of the financial departments are using it. The current temporal data tables will index only active, time crucial data, which will increase performance and old data will be in historical tables. This will give the possibility to solve any discrepancies in data or database manipulation.

3.5 Conclusion

Currently using combination of surrogate keys and 1-1 table as self referencing table. This from current research and this will change, because the sql queries are quite complex.

4 DAO versus Repository pattern

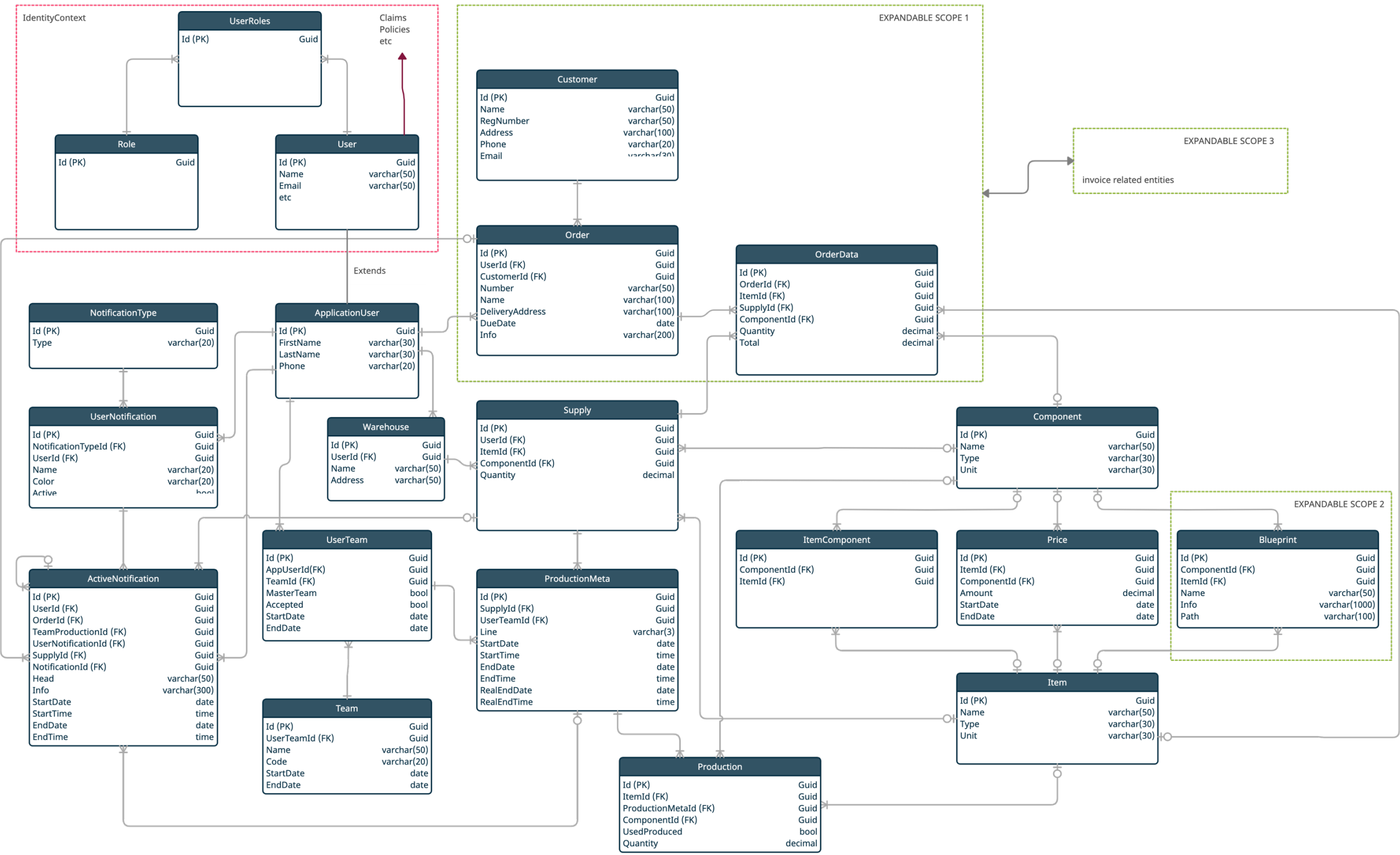
DAO (Data Access Object) and repository are patterns that as abstract layers separate business logic from database logic. Easy to change if database changes or business logic has many different database access options. Both are used for hiding data access mechanisms.

Repository pattern helps to keep the database persistence by mapping entities to relational in memory collections, hiding internal state from accessing party. It helps to avoid repetitive code in controllers and business logic. DAO on the other hand is more precise and for persistence dedicates to a single object. DAO is a class that can locate data by itself and the same for related objects where every entity has its own DAO.

In our example domain should have no outward dependencies to DAL. Meaning that every inner layer should not have dependencies to its outer layer. Repository pattern helps to achieve this by encapsulation of data process through interfaces and keeping the implementation hidden. It is usually one repository per one database table. Since Production Manager APP load needs many different user related data, some larger repositories are required, that can access multiple different data points. Repositories are wrappers around dbContext methods.

Generics in repository pattern gives a great way of reusing CRUD operations and specifying it where needed.

Annex 1 – DB Model



Annex 2